

30-DAY ROADMAP

React + Next.js + AI

Udemy Course Study Guide

Personalized for Anup Bhandari

JavaScript-first | Project-first | Full-stack + AI

TARGET 30 din pachi tutorial matra herne haina - React/Next.js concepts bujhne, full-stack app build garne, AI integrate garne, GitHub ma code push garne ra production ma deploy garne.

COURSE SNAPSHOT Verified Sep 14, 2026: 44 sections, 356 lectures, 95h 15m. Last updated Aug 2026. Course covers React, Next.js, Zustand, Prisma/Drizzle, Supabase/Convex, auth, payments, OpenAI/Gemini and production deployment.

Recommended pace: 2.5-3 hours/day

Video 60-90 min | Coding 60-75 min | Revision 15-20 min

How to use this PDF

RULE #1 Video completion bhandha skill completion important ho. Lecture herda pause garera code type garnu. Copy-paste kam garnu.

- [] Watch: aaja ko selected lectures matra herne.
- [] Understand: video banda garera concept afnai sabda ma explain garne.
- [] Code: same concept afai type garne.
- [] Mini Task: 20-40 minute ko independent task garne.
- [] GitHub: meaningful commit garne. Daily green square ko lagi matra haina - progress ko record ko lagi.
- [] Revision: day ko end ma 5 quick questions answer garne.

Priority system

Priority	Meaning	How to study
MUST	Core skill. Skip nagarne.	1x-1.25x, code along, mini task.
FAST	Tapai lai basic idea chha.	1.5x-2x, important points only.
SKIP FOR NOW	Low value/repetition for current goal.	Need paryo bhane pachi reference.

Your high-value path

React Core → Advanced React → Zustand → Advanced JS → Next.js → Database → Auth → Payments → AI → Deploy

What to fast-review or skip

- JS Refresher ko 12h+ sabai hernu pardaina. Variables, primitive basics, simple loops, Todo/Expense Tracker jasta already-known parts fast/skip.
- Queue project complete rebuild optional. Data flow, map/filter, conditional rendering ra debugging chai herne.
- Git basics fast; merge conflicts, stash, rebase, PR/open-source workflow chai MUST.
- Prisma basics fast-review; relational modeling and production usage chai revise.
- TypeScript dekhiyo bhane concept bujhne. Main practice JavaScript ma garne where practical.

30-Day at a glance

Days	Phase	Main focus	Output
1-7	React Core	Components, state, props, context, hooks, Zustand	2 mini apps + reusable store
8-14	Next.js Foundation	Advanced JS, Git, App Router, rendering, actions, middleware	Production-ready Next.js base app
15-21	Full-stack	SEO, TanStack Query, Prisma/Drizzle, Supabase/Convex, Auth	Authenticated DB-backed app
22-28	Product + AI	Payments, AI APIs, chat, advanced course projects, Next.js 16	AI feature + project architecture knowledge
29-30	Capstone	Build, test, deploy, README, portfolio	One deployed full-stack AI app

Day 1: React Foundation: Vite, JSX, Components, Props

Priority: **FAST + MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] React kina use garne, component-based architecture [] Vite setup and project structure [] JSX rules, component creation, import/export [] Props, destructuring, map() and key	[] Mini task: Developer Profile Cards: 3 profiles array bata map() use garera reusable ProfileCard render. [] Git commit: day-01-react-foundation: components, props and list rendering [] 5-minute recap: explain today's topic without notes.

Checkpoint: React ko concept code ma use garna saknu parcha - definition memorize matra hoina.

Day 2: State, Events and Forms

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] useState and reactivity [] Functional state update: setState(prev => ...) [] onClick/onChange events [] Controlled form inputs, validation, reset	[] Mini task: Counter + custom value form. Direct state mutation nagari update garne. [] Git commit: day-02-react-state: events and controlled forms [] 5-minute recap: explain today's topic without notes.

Checkpoint: React ko concept code ma use garna saknu parcha - definition memorize matra hoina.

Day 3: Data Flow + Conditional Rendering

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] Parent state, child components [] Function as props [] map/filter and immutable updates [] Conditional rendering with ternary and && [] Debugging multi-component apps	[] Mini task: Mini Queue: add person, change status, remove person, empty state. [] Git commit: day-03-react-data-flow: queue mini app [] 5-minute recap: explain today's topic without notes.

Checkpoint: React ko concept code ma use garna saknu parcha - definition memorize matra hoina.

Day 4: Advanced Props and Component Composition

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ children prop ☐ Complex/nested object props ☐ Reusable variants and default props ☐ useRef vs state ☐ forwardRef concept - focus on why/when	☐ Mini task: Reusable Modal/Card/Input components with children + ref focus button. ☐ Git commit: day-04-advanced-components: children, refs and complex props ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: React ko concept code ma use garna saknu parcha - definition memorize matra hoina.

Day 5: Context API and Global State Basics

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ Prop drilling problem ☐ createContext, Provider, useContext ☐ Theme/user shared state ☐ Context vs Zustand/Redux - when to use	☐ Mini task: ThemeContext: light/dark toggle shared across Navbar + content. ☐ Git commit: day-05-context-api: global theme state ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: React ko concept code ma use garna saknu parcha - definition memorize matra hoina.

Day 6: Custom Hooks, useEffect and Persistence

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ useEffect lifecycle mental model ☐ Dependency array and cleanup ☐ localStorage persistence ☐ Custom hook design ☐ useMemo concept ☐ storage event / cross-tab idea	☐ Mini task: useCart or useTodos custom hook with localStorage persistence. ☐ Git commit: day-06-custom-hooks: effect, persistence and reusable logic ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: React ko concept code ma use garna saknu parcha - definition memorize matra hoina.

Day 7: Zustand + React Weekly Review

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ Create store and actions ☐ Selectors to avoid unnecessary re-renders ☐ Multiple slices ☐ Async actions and API calls ☐ Context vs Zustand comparison	☐ Mini task: Zustand store: auth slice + UI slice + posts async fetch. Review Days 1-6. ☐ Git commit: day-07-zustand: slices selectors and async actions ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: React ko concept code ma use garna saknu parcha - definition memorize matra hoina.

Day 8: Advanced JavaScript Only - High Value Refresher

Priority: **MUST SELECTED LECTURES** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ Higher-order functions ☐ Closures ☐ Event loop, microtask/macrotask concept ☐ Promises and chaining ☐ async/await + Promise.all ☐ ES Modules vs CommonJS ☐ this/binding quick review	☐ Mini task: Fetch 2 APIs in parallel with Promise.all; loading/error/result handling. ☐ Git commit: day-08-advanced-js: async promises modules ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: Next.js ma server/client boundary ra routing flow clear hunu parcha.

Day 9: Git/GitHub for Real Projects

Priority: **FAST + MUST ADVANCED** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ Atomic commits and useful messages ☐ Branches and merge ☐ Resolve merge conflict ☐ git diff and stash ☐ Safe rebase concept ☐ GitHub push, PR workflow	☐ Mini task: Create feature branch, make change, merge, intentionally resolve one simple conflict. ☐ Git commit: day-09-git-workflow: branches merge and conflict practice ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: Next.js ma server/client boundary ra routing flow clear hunu parcha.

Day 10: Next.js App Router Foundation

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] Create Next.js app [] app/ folder mental model [] Server vs Client Components [] Layouts and page files [] When to add use client [] Environment variables basics	[] Mini task: Next.js dashboard shell with layout, navbar, server page and one client component. [] Git commit: day-10-nextjs-foundation: app router and server-client components [] 5-minute recap: explain today's topic without notes.

Checkpoint: Next.js ma server/client boundary ra routing flow clear hunu parcha.

Day 11: Next.js Routing Deep Dive

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] Nested routes [] Dynamic segments [] Catch-all routes [] Route groups [] loading, error and not-found [] Navigation with Link and router	[] Mini task: Blog/products routing demo with /products/[id], loading and not-found. [] Git commit: day-11-nextjs-routing: dynamic routes and special files [] 5-minute recap: explain today's topic without notes.

Checkpoint: Next.js ma server/client boundary ra routing flow clear hunu parcha.

Day 12: Data Fetching, Rendering and Caching

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] Fetch in Server Components [] SSR/SSG mental models in modern Next.js [] Caching and revalidation [] Dynamic vs static behavior [] Avoid unnecessary client fetching	[] Mini task: Products page: server fetch + revalidation + dynamic detail route. [] Git commit: day-12-nextjs-data: server fetching caching and revalidation [] 5-minute recap: explain today's topic without notes.

Checkpoint: Next.js ma server/client boundary ra routing flow clear hunu parcha.

Day 13: Server Actions and Mutations

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] Server Actions mental model [] Form action and FormData [] Validation and error states [] Create/update/delete mutation [] Revalidate after mutation	[] Mini task: Notes CRUD form with server actions (temporary data or DB if ready). [] Git commit: day-13-server-actions: forms and mutations [] 5-minute recap: explain today's topic without notes.

Checkpoint: Next.js ma server/client boundary ra routing flow clear hunu parcha.

Day 14: Route Handlers + Middleware

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] app/api route handlers [] GET/POST basics [] Request/Response and JSON [] Middleware use cases [] Protected route pattern [] Cookies/session concept	[] Mini task: Create /api/profile and middleware-protected /dashboard demo. [] Git commit: day-14-nextjs-backend: route handlers and middleware [] 5-minute recap: explain today's topic without notes.

Checkpoint: Next.js ma server/client boundary ra routing flow clear hunu parcha.

Day 15: SEO, Metadata, Performance and Deployment

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] Static/dynamic metadata [] Open Graph basics [] Image/font optimization [] Bundle/performance mindset [] Environment variables [] Vercel deployment and production checks	[] Mini task: Deploy current Next.js app. Add metadata, OG title and clean README. [] Git commit: day-15-nextjs-production: seo performance and deploy [] 5-minute recap: explain today's topic without notes.

Checkpoint: Data, auth ra backend choice ko reason explain garna saknu parcha.

Day 16: TanStack Query for Client Data

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ When TanStack Query is needed ☐ QueryClient setup ☐ useQuery loading/error/data ☐ Mutations and invalidation ☐ Server Components vs TanStack Query decision	☐ Mini task: Client-side admin list with query + create mutation + invalidate/refetch. ☐ Git commit: day-16-tanstack-query: queries mutations and cache ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: Data, auth ra backend choice ko reason explain garna saknu parcha.

Day 17: Prisma Review + Relational Modeling

Priority: **FAST REVIEW** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ Schema/model review ☐ 1-to-many relation ☐ Migrations ☐ Prisma Client usage in server code ☐ Avoid duplicate client in dev ☐ CRUD with relational data	☐ Mini task: User → Project relation. Create one user with multiple projects and fetch with relation. ☐ Git commit: day-17-prisma-review: relational data model ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: Data, auth ra backend choice ko reason explain garna saknu parcha.

Day 18: Drizzle ORM

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ Why Drizzle ☐ Schema definition ☐ Migrations ☐ Queries and relations ☐ Compare Prisma vs Drizzle	☐ Mini task: Rebuild a tiny Notes table using Drizzle; document Prisma vs Drizzle differences. ☐ Git commit: day-18-drizzle: schema migrations and queries ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: Data, auth ra backend choice ko reason explain garna saknu parcha.

Day 19: Supabase Masterclass Focus

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ Supabase project + database ☐ Client/server usage ☐ Auth overview ☐ Storage concept ☐ RLS concept - understand before production	☐ Mini task: Supabase-backed simple notes/materials list. Read/write from Next.js. ☐ Git commit: day-19-supabase: database and app integration ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: Data, auth ra backend choice ko reason explain garna saknu parcha.

Day 20: Convex Masterclass Focus

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ Convex mental model ☐ Queries and mutations ☐ Realtime updates ☐ Schema/data model ☐ When Convex is better/worse than SQL stack	☐ Mini task: Realtime task list or chat messages with Convex. ☐ Git commit: day-20-convex: realtime queries and mutations ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: Data, auth ra backend choice ko reason explain garna saknu parcha.

Day 21: Authentication: Better Auth, Clerk and OAuth

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ Session/auth flow ☐ Protected routes ☐ Google/GitHub OAuth concept ☐ Better Auth setup focus ☐ Clerk setup overview ☐ Store user identity with app data	☐ Mini task: Add login + protected dashboard using ONE provider fully. Compare second provider conceptually. ☐ Git commit: day-21-auth: login session and protected dashboard ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: Data, auth ra backend choice ko reason explain garna saknu parcha.

Day 22: Payments: Stripe, Polar, Razorpay

Priority: **MUST CONCEPT + ONE IMPLEMENTATION** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ Checkout/payment flow ☐ Server-side payment creation ☐ Webhook concept ☐ Success/cancel handling ☐ Never trust client payment state ☐ Compare gateway use cases	☐ Mini task: Implement test checkout with one gateway or build complete mocked payment flow if account setup blocks you. ☐ Git commit: day-22-payments: checkout and webhook architecture ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: Feature ko architecture bujhne; tutorial ko exact code copy nagarne.

Day 23: AI Integration: OpenAI/Gemini Securely

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ API key only on server ☐ Route/server action for AI call ☐ Prompt structure ☐ Streaming concept ☐ Error/rate-limit mindset ☐ Save AI output to DB when useful	☐ Mini task: AI Text Helper: user prompt → server → model → rendered response. ☐ Git commit: day-23-ai-integration: secure server-side model call ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: Feature ko architecture bujhne; tutorial ko exact code copy nagarne.

Day 24: AI Chat App Architecture

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
☐ Message data model ☐ User/assistant roles ☐ Conversation persistence ☐ Streaming UI concept ☐ Loading/error/retry ☐ Context window awareness	☐ Mini task: Build minimal persistent AI chat with conversation list and message history. ☐ Git commit: day-24-ai-chat: persistent conversation flow ☐ 5-minute recap: explain today's topic without notes.

Checkpoint: Feature ko architecture bujhne; tutorial ko exact code copy nagarne.

Day 25: Dashboard, File Handling and AI Assistant Patterns

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] Dashboard component structure [] File upload architecture [] Validation and size/type checks [] Store file metadata [] AI processing pipeline concept	[] Mini task: Upload UI + source list + AI action button. Use safe mock processing if parser is not ready. [] Git commit: day-25-ai-product-patterns: files dashboard and processing flow [] 5-minute recap: explain today's topic without notes.

Checkpoint: Feature ko architecture bujhne; tutorial ko exact code copy nagarne.

Day 26: LeetCode Clone - Architecture Study

Priority: **MUST PROJECT STUDY** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] Break large app into modules [] Problem/detail/editor/result flow [] Database entities and relations [] Submission lifecycle [] Auth and user progress	[] Mini task: Do not blindly copy. Draw architecture and rebuild one core flow (problem list → detail → submit mock). [] Git commit: day-26-project-architecture: coding platform core flow [] 5-minute recap: explain today's topic without notes.

Checkpoint: Feature ko architecture bujhne; tutorial ko exact code copy nagarne.

Day 27: V0 Builder Clone - Generative Product Study

Priority: **MUST PROJECT STUDY** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
[] Prompt → generation → preview flow [] Component/code generation architecture [] State and version history idea [] Security: never execute untrusted code blindly [] Iterative prompt UX	[] Mini task: Build safe UI prototype: prompt history + generated static component preview using predefined templates. [] Git commit: day-27-generative-builder: prompt-to-preview prototype [] 5-minute recap: explain today's topic without notes.

Checkpoint: Feature ko architecture bujhne; tutorial ko exact code copy nagarne.

Day 28: Next.js 16 + Production Patterns

Priority: **MUST** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
Course Next.js 16 updates/masterclass Review current APIs used in course Error handling and logging Security checklist Caching/performance revisit Production env sanity	Mini task: Audit your project: server/client boundaries, env vars, errors, caching, auth and performance. Git commit: day-28-production-audit: nextjs patterns and security review 5-minute recap: explain today's topic without notes.

Checkpoint: Feature ko architecture bujhne; tutorial ko exact code copy nagarne.

Day 29: Capstone Build Day 1 - Your Own AI SaaS

Priority: **BUILD** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
Pick ONE narrow problem Define user, pain, promise 3 core features only Choose stack: Next.js + one DB + one auth + one AI provider Design schema and routes before coding	Mini task: Suggested: AI Study Assistant - login, create subject, ask AI, save chat/notes. Build auth + DB + core screens. Git commit: day-29-capstone: core product auth db and ui 5-minute recap: explain today's topic without notes.

Checkpoint: Scope sano rakhera complete, deployed, testable product ship garne.

Day 30: Capstone Build Day 2 - Ship It

Priority: **BUILD + SHIP** | Target time: 2.5-3 hours

WATCH + UNDERSTAND	CODE + OUTPUT
Finish one AI workflow end-to-end Add validation/loading/error states Deploy production Test mobile + desktop README with screenshots/features/setup Portfolio/LinkedIn-ready summary	Mini task: Deploy final app. Ask 1-2 people to test. Record bugs, fix critical ones, publish GitHub README. Git commit: day-30-capstone: ai workflow deploy and launch 5-minute recap: explain today's topic without notes.

Checkpoint: Scope sano rakhera complete, deployed, testable product ship garne.

30-Day Completion Checklist

- ☐ React components, props, state, forms and conditional rendering confidently use garna sakchu.
 - ☐ Context ra Zustand ko difference bujhera appropriate option choose garna sakchu.
 - ☐ useEffect, cleanup, localStorage and custom hooks use garna sakchu.
 - ☐ Promises, async/await, Promise.all and modules clear chha.
 - ☐ Git branch, merge conflict, stash, rebase concept and PR workflow practice gareko chu.
 - ☐ Next.js App Router, dynamic routes, server/client components clear chha.
 - ☐ Data fetching, caching/revalidation and server actions use gareko chu.
 - ☐ Route handlers and middleware bata backend/protection setup gareko chu.
 - ☐ Prisma revise gareko chu; Drizzle, Supabase and Convex ko tradeoffs bujheko chu.
 - ☐ Authentication provider use garera protected dashboard banayeko chu.
 - ☐ Payment architecture/webhook concept bujheko chu.
 - ☐ AI API server-side secure route bata integrate gareko chu.
 - ☐ At least one AI chat or AI workflow end-to-end build gareko chu.
 - ☐ Production deployment gareko chu.
 - ☐ GitHub README + portfolio-ready project chha.
- PASS CONDITION** 30/30 video streak matra pass condition hoina. Kam se kam 1 deployed full-stack AI app, 20+ meaningful commits, ra core concepts afnai sabda ma explain garna saknu is the real target.

Daily study formula

Block	Time	Action	Rule
1	60-90 min	Udemy lectures	Pause and type code.
2	60-75 min	Rebuild/practice	Do not look at solution first.
3	15-20 min	Notes + 5 questions	Write only what you can reuse.
4	5-10 min	Git commit	One meaningful checkpoint.

After Day 30: What next?

- Week 5-6: Choose one serious product and improve it instead of starting another tutorial.
- Add tests, analytics, error monitoring, rate limiting and better UX.
- Do one open-source PR or team project to practice real Git workflow.
- Prepare one strong case study: problem → architecture → challenges → solution → demo link.
- Then explore mobile (React Native/Expo) only after your web product workflow feels comfortable, unless mobile is the product requirement.

Course source note

Plan based on the Udemy course "Complete React and NextJS course with AI powered Projects" by Hitesh Choudhary and Suraj Kumar Jha. Course page verified on Sep 14, 2026: last updated Aug 2026; 44 sections, 356 lectures, 95h 15m. The roadmap is intentionally personalized and does not require watching every lecture in sequence.